IT Workshop II (Cloud Computing): Hadoop - Concepts, Cloud Security

October 10, 2026

Resilient Distributed Datasets (RDDs)

- RDDs (Resilient Distributed Datasets) is Data Containers
- All the different processing components in Spark share the same abstraction called RDD
- As applications share the RDD abstraction, you can mix different kind of transformations to create new RDDs
- There are three ways of creating RDDs: from an in-memory collection of objects (known as parallelizing a collection), using a dataset from external storage (such as HDFS), or transforming an existing RDD.
- **Example:**
  val params = sc.parallelize(1 to 10)
  val result = params.map(performExpensiveComputation)
  It runs separate computations on the numbers from 1 to 10.

Transformations and Actions

- Spark provides two categories of operations on RDDs: transformations and actions.
- A transformation generates a new RDD from an existing one, while an action triggers a computation on an RDD and does something with the results - either returning them to the user, or saving them to external storage.
- Actions have an immediate effect, but transformations do not - they are lazy, in the sense that they don't perform any work until an action is performed on the transformed RDD.

Transformations and Actions

- For example, the following lowercases lines in a text file:
  val text = sc.textFile(inputPath)
  val lower: RDD[String] = text.map(_.toLowerCase())
  lower.foreach(println(_))
  The map() method is a transformation, which Spark represents internally as a function (toLowerCase()) to be called at some later time on each element in the input RDD (text). The function is not actually called until the foreach() method (which is an action) is invoked and Spark runs a job to read the input file and call toLowerCase() on each line in it, before writing the result to the console.
- If the return type is RDD, then it is a transformation; otherwise, it is an action.

Transformations and Actions

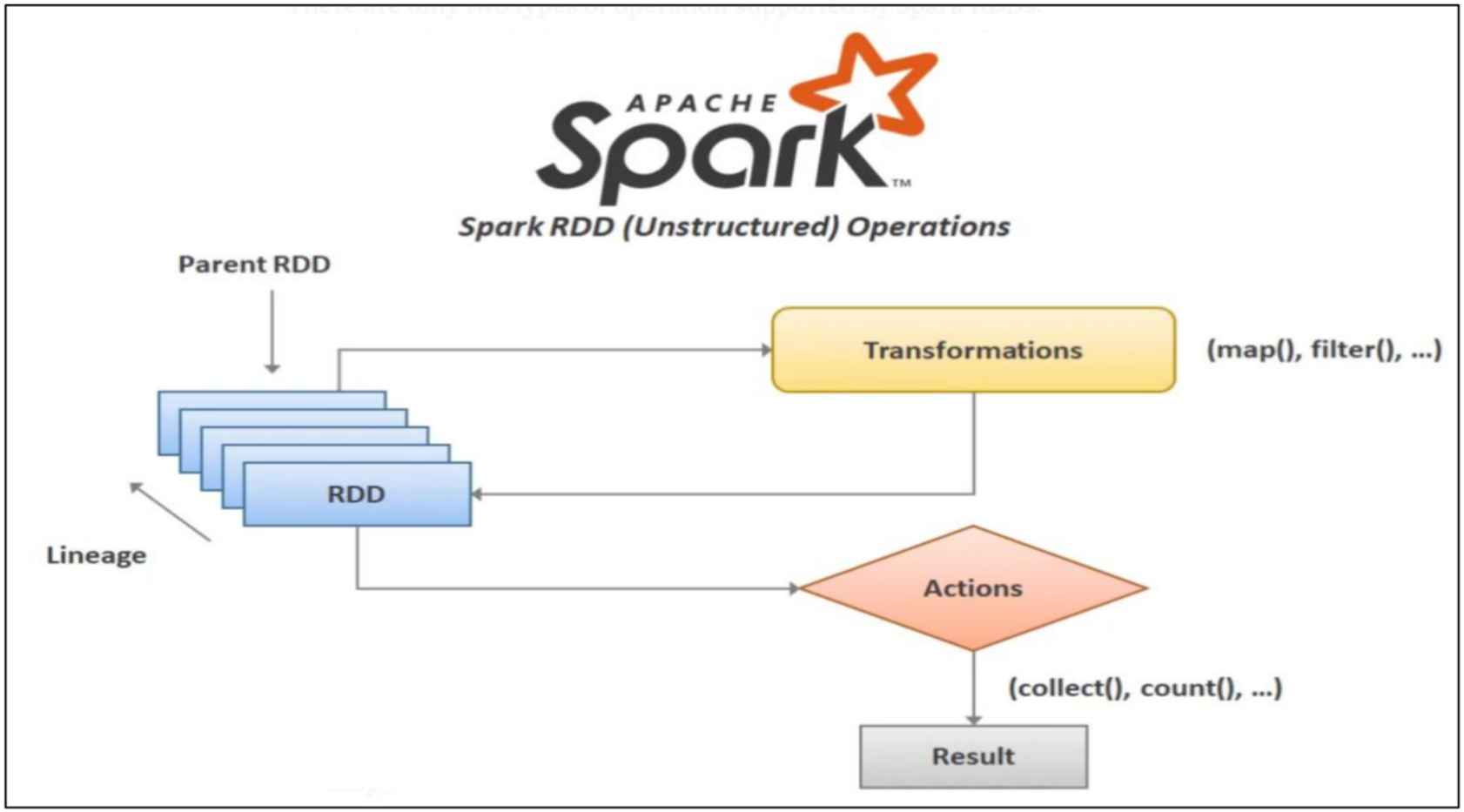


Figure: Source: Internet

Some Transformations

Transformation	Meaning
map(*func*)	Return a new distributed dataset formed by passing each element of the source through a function *func*.
filter(*func*)	Return a new dataset formed by selecting those elements of the source on which *func* returns true.
flatMap(*func*)	Similar to map, but each input item can be mapped to 0 or more output items (so *func* should return a Seq rather than a single item).
mapPartitions(*func*)	Similar to map, but runs separately on each partition (block) of the RDD, so *func* must be of type Iterator<T> => Iterator<U> when running on an RDD of type T.
mapPartitionsWithIndex(*func*)	Similar to mapPartitions, but also provides *func* with an integer value representing the index of the partition, so *func* must be of type (Int, Iterator<T>) => Iterator<U> when running on an RDD of type T.

Figure: Source: Internet

Some Actions

Action	Meaning
reduce(*func*)	Aggregate the elements of the dataset using a function *func* (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
collect()	Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
count()	Return the number of elements in the dataset.
first()	Return the first element of the dataset (similar to take(1)).
take(*n*)	Return an array with the first *n* elements of the dataset.
takeSample(*withReplacement*, *num*, [*seed*])	Return an array with a random sample of *num* elements of the dataset, with or without replacement, optionally pre-specifying a random number generator seed.

Figure: Source: Internet

Cloud Security Challenges

- **Authentication:**
  - Authentication refers to digitally confirming the identity of the entity requesting access to some protected information.
  - In cloud computing environments, where applications and data are accessed over the internet, the complexity of digital authentication mechanisms increases rapidly.
- **Authorization:**
  - Authorization refers to digitally specifying the access rights to the protected resources using access policies.
  - Authorization in a cloud computing environment requires the use of the cloud service providers services for specifying the access policies.

Cloud Security Challenges

- **Security of data at rest:**
  - Due to the multi-tenant environments used in the cloud, the application and database servers of different applications belonging to different organizations can be provisioned side-by-side increasing the complexity of securing the data.
  - Appropriate separation mechanisms are required to ensure the isolation between applications and data from different organizations.
- **Security of data in motion:**
  - With the adoption of the cloud model, the applications and the data are moved out of the in-house IT infrastructure to the cloud provider.
  - Therefore, appropriate security mechanisms are required to ensure the security of data in, and while in, motion.

Cloud Security Challenges

- **Data Integrity:** Data integrity ensures that the data is not altered in an unauthorized manner after it is created, transmitted or stored. Due to the outsourcing of data storage in cloud computing environments, ensuring integrity of data is important.
- **Auditing:** For cloud applications appropriate auditing mechanisms are required to get visibility into the application, data accesses and actions performed by the application users, including mobile users and devices such as wireless laptops and smartphones.

Authentication

- The process of authentication involves, but is not limited to, validating the at least one factor of identification of the entity to be authenticated.
- A factor can be something the entity or the user knows (password or pin), something the user has (such as a smart card), or something that can uniquely identify the user (such as fingerprints).
- In multifactor authentication more than one of these factors are used for authentication.
- There are various mechanisms for authentication including:
  - SSO
  - SAML-Token
  - OTP

Authentication

- Single Sign-on (SSO) enables users to access multiple systems or applications after signing in only once, for the first time.
- When a user signs in, the user identity is recognized and there is no need to sign in again and again to access related systems or applications.
- The benefit of using SSO is that it reduces human error and saves time spent in authenticating with different systems or applications for the same identity.

The End